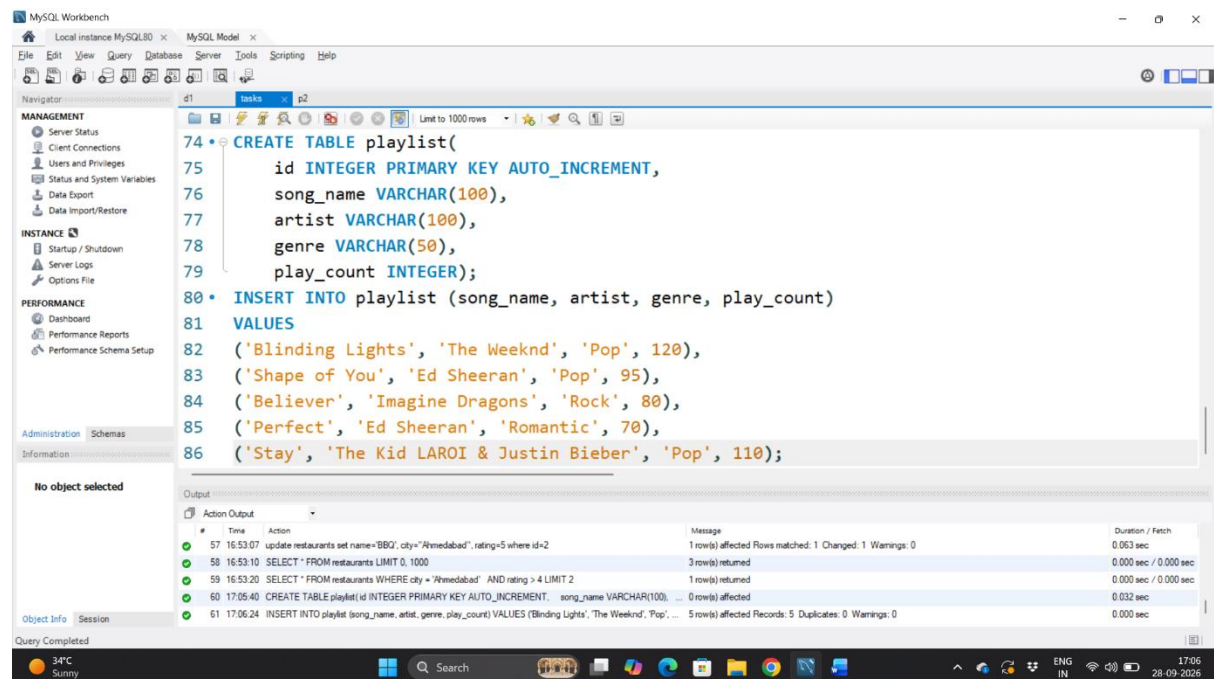


Session 4

Q1. Create a table called Playlist with columns: id, song_name, artist, genre, and play_count. Insert at least 5 sample records representing your favorite songs from Spotify.

Ans:

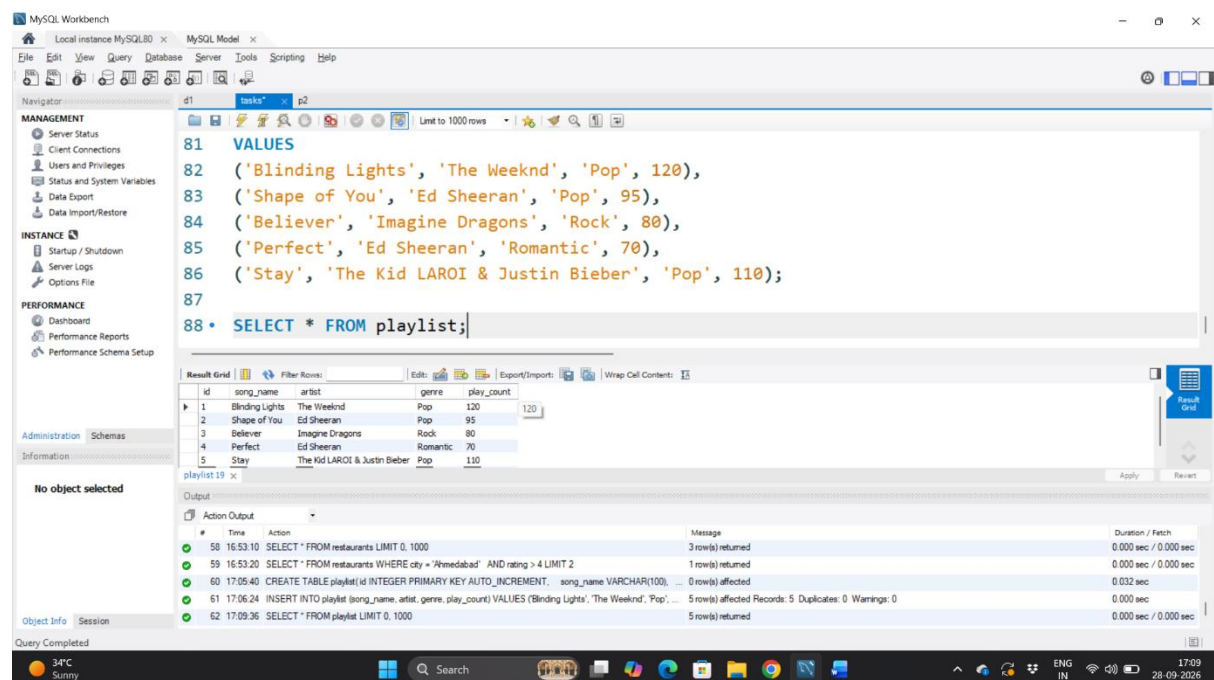


The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
74 CREATE TABLE playlist(  
75     id INTEGER PRIMARY KEY AUTO_INCREMENT,  
76     song_name VARCHAR(100),  
77     artist VARCHAR(100),  
78     genre VARCHAR(50),  
79     play_count INTEGER);  
80 INSERT INTO playlist (song_name, artist, genre, play_count)  
81 VALUES  
82 ('Blinding Lights', 'The Weeknd', 'Pop', 120),  
83 ('Shape of You', 'Ed Sheeran', 'Pop', 95),  
84 ('Believer', 'Imagine Dragons', 'Rock', 80),  
85 ('Perfect', 'Ed Sheeran', 'Romantic', 70),  
86 ('Stay', 'The Kid LAROI & Justin Bieber', 'Pop', 110);
```

The Output tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
57	16:53:07	update restaurants set name='BBQ', city='Ahmedabad', rating=5 where id=2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.063 sec
58	16:53:10	SELECT * FROM restaurants LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
59	16:53:20	SELECT * FROM restaurants WHERE city = 'Ahmedabad' AND rating > 4 LIMIT 2	1 row(s) returned	0.000 sec / 0.000 sec
60	17:05:40	CREATE TABLE playlist(id INTEGER PRIMARY KEY AUTO_INCREMENT, song_name VARCHAR(100),	0 row(s) affected	0.032 sec
61	17:06:24	INSERT INTO playlist (song_name, artist, genre, play_count) VALUES ('Blinding Lights', 'The Weeknd', 'Pop',	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
81 VALUES  
82 ('Blinding Lights', 'The Weeknd', 'Pop', 120),  
83 ('Shape of You', 'Ed Sheeran', 'Pop', 95),  
84 ('Believer', 'Imagine Dragons', 'Rock', 80),  
85 ('Perfect', 'Ed Sheeran', 'Romantic', 70),  
86 ('Stay', 'The Kid LAROI & Justin Bieber', 'Pop', 110);  
87  
88 SELECT * FROM playlist;
```

The Output tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
58	16:53:10	SELECT * FROM restaurants LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
59	16:53:20	SELECT * FROM restaurants WHERE city = 'Ahmedabad' AND rating > 4 LIMIT 2	1 row(s) returned	0.000 sec / 0.000 sec
60	17:05:40	CREATE TABLE playlist(id INTEGER PRIMARY KEY AUTO_INCREMENT, song_name VARCHAR(100),	0 row(s) affected	0.032 sec
61	17:06:24	INSERT INTO playlist (song_name, artist, genre, play_count) VALUES ('Blinding Lights', 'The Weeknd', 'Pop',	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
62	17:09:36	SELECT * FROM playlist LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

The Result Grid shows the following data:

	id	song_name	artist	genre	play_count
1	1	Blinding Lights	The Weeknd	Pop	120
2	2	Shape of You	Ed Sheeran	Pop	95
3	3	Believer	Imagine Dragons	Rock	80
4	4	Perfect	Ed Sheeran	Romantic	70
5	5	Stay	The Kid LAROI & Justin Bieber	Pop	110

Q2. Write a SELECT query to display only the song_name and artist from the Playlist table, but show the artist column as 'Singer' using an alias.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```

86 ('Stay', 'The Kid LAROI & Justin Bieber', 'Pop', 110);
87
88 SELECT * FROM playlist;
89
90
91 SELECT song_name, artist AS Singer FROM playlist;

```

The Results grid shows the output of the first query (SELECT * FROM playlist):

song_name	artist
Blinding Lights	The Weeknd
Shape of You	Ed Sheeran
Believer	Imagine Dragons
Perfect	Ed Sheeran
Stay	The Kid LAROI & Justin Bieber

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
60	17:05:40	CREATE TABLE playlist (id INTEGER PRIMARY KEY AUTO_INCREMENT, song_name VARCHAR(100), ...	0 rows(s) affected	0.032 sec
61	17:06:24	INSERT INTO playlist (song_name, artist, genre, play_count) VALUES ('Blinding Lights', 'The Weeknd', 'Pop', ...	5 rows(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
62	17:09:36	SELECT * FROM playlist LIMIT 0, 1000	5 rows(s) returned	0.000 sec / 0.000 sec
63	17:12:45	SELECT song_name, artist AS Singer FROM playlist LIMIT 0, 1000	5 rows(s) returned	0.000 sec / 0.000 sec
64	17:13:03	SELECT song_name, artist AS Singer FROM playlist LIMIT 0, 1000	5 rows(s) returned	0.000 sec / 0.000 sec

Q3. Write a SELECT query to find all songs in the Playlist table where the genre is 'Pop' and play_count is greater than 100. Sort the results by play_count in descending order.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

92
93 SELECT *
94 FROM Playlist
95 WHERE genre = 'Pop'
96 AND play_count > 100
97 ORDER BY play_count DESC;

```

The Results grid shows the output of the query:

id	song_name	artist	genre	play_count
1	Blinding Lights	The Weeknd	Pop	120
5	Stay	The Kid LAROI & Justin Bieber	Pop	110

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
64	17:13:03	SELECT song_name, artist AS Singer FROM playlist LIMIT 0, 1000	5 rows(s) returned	0.000 sec / 0.000 sec
65	17:18:47	SELECT song_name, genre, play_count where genre='Pop' and play_count>100 from playlist	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.000 sec
66	17:18:58	SELECT song_name, genre, play_count where genre='Pop'; play_count>100 from playlist	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.000 sec
67	17:19:12	SELECT song_name, genre, play_count where genre='Pop' play_count>100 from playlist	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.000 sec
68	17:19:58	SELECT * FROM Playlist WHERE genre = 'Pop' AND play_count > 100 ORDER BY play_count DESC LIMIT...	2 rows(s) returned	0.000 sec / 0.000 sec

Q4. Use the COUNT aggregate function to find out how many songs in your Playlist table belong to the genre 'Hip-Hop'.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
95 WHERE genre = 'Pop'
96 AND play_count > 100
97 ORDER BY play_count DESC;
98
99. select count(song_name) from playlist where genre='Hip-Hop';
```

The Results panel shows a single row with the count of songs for the 'Hip-Hop' genre:

count(song_name)
0

The Action Output panel shows the execution of the query, with a message indicating an error: "Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'select count(song_name) from playlist where genre='Hip-Hop';' at line 99."

Q5. Write a SELECT query that shows the total number of plays (SUM of play_count) for each genre in your Playlist table, grouped by genre. Hint: Use GROUP BY with the SUM function to aggregate play counts by genre.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
96 AND play_count > 100
97 ORDER BY play_count DESC;
98
99. select count(song_name) from playlist where genre='Hip-Hop';
00
01. SELECT genre, sum(play_count) as total_plays from playlist GROUP BY genre;
```

The Results panel shows the execution of the query, with a message indicating an error: "Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'select count(song_name) from playlist where genre='Hip-Hop';' at line 99."

The Action Output panel shows the execution of the query, with a message indicating an error: "Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'select count(song_name) from playlist where genre='Hip-Hop';' at line 99."